

تقرير أعمال Abdulelah في مشروع FoxFlow

مرجع تنفيذي وتقني لأعمال العضو الرابع على الجهاز والخادم



البيان	القيمة
الاسم	Abdulelah Alshaya
الدور	DevSecOps والتكامل التشغيلي CI/CD
فترة العمل الموثقة	15 إلى 27 سبتمبر 2026
آخر تشغيل موثق	GitLab Pipeline #44
الحالة النهائية	مكتمل وجاهز للعرض

يوثق هذا التقرير الأعمال التي نفذتها أو تابعتها من جهاز Mac ضمن مشروع FoxFlow، بما في ذلك التعديلات المحلية، إعداد GitLab و Azure، تصميم مسار CI/CD، الأمان، المراقبة، التنبيهات، النسخ الاحتياطي، لوحة العمليات، HTTPS، تجربة الفشل، التوثيق، وتجهيز العرض النهائي. لا يتضمن التقرير كلمات مرور أو مفاتيح أو رموز وصول.

الملخص التنفيذي

توليت الجزء الذي يحول الكود إلى نسخة قابلة للبناء والاختبار والفحص الأمني والنشر والمراقبة. أصبحت GitLab منصة العمل الرئيسية، وبدأ المسار عند دفع التغيير إلى الفرع المحمي ثم ينفذ GitLab Runner البناء والاختبارات وفحص Trivy والنشر إلى Azure، وبعدها يرسل النتيجة إلى Telegram ويعرضها في لوحة FoxFlow.

النتيجة النهائية: التطبيق وGitLab يعملان عبر HTTPS، آخر Pipeline موثق هو رقم 44، جميع الوظائف المطلوبة ناجحة، الاختبارات 16 من 16، فحص Trivy لم يجد ثغرات High أو Critical تمنع النشر، وحماية التطبيق تعمل بحالة healthy. النسخ الاحتياطي اليومي موجود في Azure Blob وتم تنزيل أحدث نسخة وفحص سلامة أرشيفها بنجاح.

الحالة	الدليل	البند
مكتمل	Pipeline #44 وجميع وظائفه ناجحة	CI/CD
مكتمل	16/16 اختبارًا ناجحًا	الاختبارات
مكتمل	0 High أو Critical في بوابة Trivy	الأمن
مكتمل	HTTP 200 وJSON صحيح من health/	النشر
مكتمل	Telegram وOpenRouter وتحليل محلي بديل	التنبهات الذكية
مكتمل	نسخة 26 سبتمبر اجتازت tar -tf	النسخ الاحتياطي
مكتمل	HTTPS للتطبيق وGitLab	الاتصال الآمن

نطاق دوري في المشروع

- تصميم GitLab CI/CD وربط المراحل ببعضها ومنع النشر عند فشل أي بوابة إلزامية.
- ربط GitLab Runner وDocker in Docker مع GitLab Container Registry الداخلي.
- تشغيل اختبارات التطبيق واختبارات AI Copilot داخل Pipeline.
- فحص صورة Docker بواسطة Trivy وحجب الثغرات العالية والحرية.
- النشر الآلي إلى Azure VM عبر SSH وDocker Compose والتحقق من صحة التطبيق.
- إنشاء إشعارات نجاح وفشل عبر Telegram وإضافة تحليل OpenRouter للأخطاء.
- إضافة مراقبة استباقية للخدمات والقرص وحدثة النسخ الاحتياطية.
- إنشاء لوحة FoxFlow Control Room وربطها ببيانات GitLab الحقيقية وحمايتها بحسابات أعضاء الفريق.
- تفعيل HTTPS للتطبيق وGitLab باستخدام Caddy مع إبقاء المنافذ الداخلية غير عامة.
- تجهيز AI DevOps Copilot في Telegram بأدوات محددة وآمنة.
- تجهيز تجربة فشل معزولة أمام اللجنة لا تؤثر في الإنتاج.
- مراجعة المشروع والتوثيق والعرض التقديمي ومطابقة الأرقام مع التشغيل الفعلي.

التسلسل الزمني للأعمال

الفترة	النتيجة	العمل المنفذ
15 إلى 17 سبتمبر	تصميم واضح لمسؤولية العضو الرابع	تحديد مراحل CI/CD ومتطلبات التطبيق والـ Runner والنشر، وتجهيز اختبارات Compose محلية.

نشر sample-app إلى GitLab، وضبط Registry الداخلي، ومعالجة BuildKit وTrivy وصورة التشغيل.	أول Pipeline كامل	20 سبتمبر
تفعيل deploy والتحقق من health/، ثم إضافة الواجهة والمراقبة وTelegram وتجربة الفشل المعزولة.	إنتاج يعمل على Azure	21 سبتمبر
إضافة تحليل AI للفشل، تحسين رسائل Telegram، إنشاء لوحة Control Room وربطها ببيانات GitLab الحية.	تجربة عرض أكثر وضوحًا	22 إلى 25 سبتمبر
فصل الصفحة العامة عن لوحة العمليات، إنشاء حسابات، إضافة Copilot، نقل الحالة والصلاحيات، وتفعيل HTTPS.	حماية وتكامل متقدم	25 إلى 26 سبتمبر
مخططات معمارية حديثة، مراجعة العرض، تحديث الأدلة إلى Pipeline #44 والنسخة الاحتياطية الأحدث.	توثيق وتسليم	26 إلى 27 سبتمبر

ما نفذته على جهاز Mac

- إدارة نسخة المشروع المحلية داخل SDA-project-FoxFlow ومراجعة حالة Git والملفات قبل الدفع.
- تعديل ملفات التطبيق وDocker وCI/CD والسكريبتات والوثائق، ثم تشغيل اختبارات محلية قبل النشر.
- استخدام مفتاح SSH مخصص للاتصال بـGitLab والخادم، مع عدم مشاركة المفتاح الخاص بين الأعضاء.
- تشغيل publish-sample.sh لنشر محتويات sample-app إلى مشروع GitLab الأساسي.
- التحقق من Azure CLI والاشتراك الصحيح بعد اكتشاف اشتراك آخر معطل بالاسم نفسه.
- فحص التطبيق من خارج الخادم باستخدام curl، ومراجعة GitLab Pipelines وJobs من المتصفح.
- إنشاء وتحديث التقرير والمخططات والعرض التقديمي في Canva ومطابقته مع النتائج الحقيقية.

ما نفذته أو تحققت منه على Azure وGitLab

- تشغيل GitLab CE وRunner وRegistry والتطبيق داخل حاويات Docker على Azure VM.
- إضافة متغيرات CI/CD المحمية اللازمة للنشر وTelegram وOpenRouter واللوحة.
- التحقق من حماية main ومنع النشر إلا من الفرع الافتراضي المحمي.
- تثبيت وتشغيل خدمات المراقبة وAI Copilot بواسطة systemd.
- ضبط Caddy كـTLS reverse proxy للتطبيق وGitLab.
- التأكد من النسخ اليومية إلى Azure Blob ومن سلامة أحدث ملف backup.
- إنشاء حسابات الوصول وحماية لوحة العمليات مع جلسات Cookies آمنة.

البنية النهائية وطريقة تدفق التغيير

GitLab هو مساحة العمل الرئيسية للكود والمراجعات وتشغيل Pipeline. يحتفظ GitHub بنسخة أرشيفية من ملفات المشروع. عند وصول التغيير إلى main، ينفذ Runner المراحل الخمس، بينما توجد عدة وظائف داخل مرحلة الاختبار والإشعار.

الوجهة	التدفق
GitLab	المطور يدفع التعديل أو يدمج Merge Request إلى main المحمي
Runner	Build ثم Test و Copilot Test ثم Security Scan
Registry	تخزين صورة Docker باسم مرتبط بمعرف commit الكامل
Azure VM	Deploy عبر SSH و Docker Compose ثم انتظار healthcheck
Operations	Dashboard ومراقبة وTelegram وتحليل AI ونسخ Azure
GitHub	نسخة أرشيفية ووثائق ومخططات المشروع

```
Developer -> GitLab (primary) -> Runner
-> build -> test + copilot_test -> Trivy
-> Registry -> SSH deploy -> Azure VM
-> health check -> Dashboard -> Telegram / AI
GitHub = archive copy
```

المخطط المعماري التفصيلي موجود داخل docs، ويعرض GitLab كمنصة أساسية، Registry داخليًا، Caddy وHTTPS، تطبيق FoxFlow، المراقبة، Telegram، OpenRouter، و Azure Blob Backup.

مراحل CI/CD التي أنشأتها

المرحلة	ما يحدث	شرط النجاح
Build	بناء صورة من Dockerfile ورفعها باسم CI_COMMIT_SHA\$	نجاح docker build و push
Test	npm ci و npm test واختبارات AI Copilot	16/16 اختبارًا ناجحًا
Security Scan	فحص الصورة البعيدة وحفظ -trivy report.json	لا High أو Critical
Deploy	نشر عبر SSH و Compose على Azure من main المحمي	حالة healthy و HTTP 200
Notify	notify_success أو notify_failure مع رابط Pipeline	وصول الرسالة أو تحذير فقط

مرحلة Deploy لا تظهر إلا عندما يكون الفرع هو الفرع الافتراضي والمحمي وتكون DEPLOY_ENABLED=true. كما استخدمت resource_group لمنع تداخل عمليتي نشر في الوقت نفسه، والتحقق الصارم من SSH known_hosts بدل تعطيل فحص هوية الخادم.

الاختبارات وبوابات الجودة

رفعت الاختبارات من فحوص بسيطة إلى مجموعة تغطي التطبيق ولوحة العمليات ومخرجات API وحماية جلسات الدخول وسلوك AI Copilot. أحدث تحقق محلي وداخل Pipeline سجل 12 اختبار Node.js و4 اختبارات Python، أي 16 من 16.

العدد	أمثلة على التحقق	المجموعة
12	health/ و HTML و 404 و 405 والمصادقة و API أمن وملخص Pipeline منقح	Node.js
4	رفض الأدوات غير المسموحة وحجب الأسرار والتحليل المحلي الأمن	AI Copilot
16/16	لا فشل في آخر تحقق	الإجمالي

الأمن الذي أضعفته

- Trivy يمنع النشر عند وجود ثغرات High أو Critical قابلة للإصلاح.
- صورة التشغيل تستخدم حساب node غير الجذر، وحُذفت منها npm و npx لتقليل سطح الهجوم.
- Registry يعمل داخل الشبكة الخاصة ولا يحتاج إلى كشف المنفذ 5050 للعامة.
- مفاتيح SSH و known_hosts تُستخدم كمتغيرات File محمية ولا تُحفظ في Git.
- Telegram و OpenRouter tokens محفوظة كمتغيرات Masked و Protected ولا تظهر في السجلات.
- التسجيل العام في GitLab مغلق، و 2FA إلزامية، والمشروعات الجديدة Private افتراضياً.
- لوحة العمليات تتطلب حساباً مستقلاً، وكلمات المرور مخزنة كـ script hashes.
- الجلسات تستخدم Cookies موقعة و Secure و HTTP-only.
- Caddy يوفر HTTPS، بينما تبقى المنافذ 3000 و 5050 و 8080 و 8443 خاصة داخل NSG.

لوحة FoxFlow Control Room

أنشأت لوحة مرئية حتى يتمكن الفريق واللجنة من معرفة حالة المشروع دون فتح صفحات GitLab المتعددة. يقرأ الخادم بيانات GitLab بواسطة token للقراءة فقط، وينقح المخرجات قبل إرسالها للمتصفح.

- آخر Pipeline وحالتها ورقمها ومدتها.
- حالات Build و Test و Security و Deploy و Notify.
- حالة Runner و Trivy ونتيجة النشر.
- ملخص فشل AI عند توفره وسجل النشاط.
- صفحة عامة منفصلة عن لوحة العمليات المحمية.

Telegram والتحليل الذكي

إشعارات Pipeline

أضفت notify_success و notify_failure. رسالة النجاح تعرض المشروع والفرع والـ commit و رابط Pipeline. عند الفشل، تحاول الوظيفة قراءة المهمة الفاشلة وسجلها ثم ترسل الخطأ إلى OpenRouter للحصول على Cause و Evidence و Suggested fix. إذا فشل API أو أعاد تنسيقاً غير صالح، يستخدم النظام تحليلاً محلياً بدلاً إرسال رسالة فارغة أو مضللة.

المراقبة الاستباقية

تُثبت خدمة systemd مع timer لفحص البيئة دوريًا. تبقى صامتة عند ثبات الحالة، وترسل فقط عند ظهور مشكلة أو حدوث تعاف حتى لا تمتلئ المجموعة برسائل متكررة.

السلوك	الفحص
تشغيل الحاوية وصحتها	GitLab
حالة Runner	Runner
تشغيل الحاوية وصحتها	Application
استجابة صفحة الدخول	GitLab HTTP
HTTP 200 والجسم المتوقع	Application health
تحذير عند 75% وفشل عند 90%	Disk
تحذير بعد 26 ساعة وفشل بعد 36 ساعة	Backup freshness

AI DevOps Copilot

ربطت Telegram بـ OpenRouter وأدوات تشغيل محددة بدل السماح للذكاء الاصطناعي بتنفيذ أوامر حرة. يفهم البوت الأسئلة العربية والإنجليزية ويرد بالإنجليزية حسب قرار الفريق.

الغرض	الأداة المسموحة
فحص GitLab و Runner والتطبيق والقرص والنسخ	check_status
تشغيل نسخة احتياطية محمية	run_backup
قراءة آخر Pipeline من GitLab API	get_pipeline
قراءة آخر 50 سطرًا من سجلات محددة	read_logs
إعادة تشغيل خدمة من قائمة محمية	restart_service

تمت إضافة اختبارات تمنع اختيار أداة غير معروفة، أو خدمة غير مسموحة، أو تسريب قيمة تشبه secret. كما نُقلت ملكية ملفات الحالة ومسارات التشغيل إلى حساب الخدمة المناسب وصلاحيات مقيدة.

HTTPS والشبكة

انتقل المشروع من عناوين HTTP ومنافذ ظاهرة إلى دخول موحد عبر HTTPS. يستخدم Caddy شهادات TLS ويعمل كـ reverse proxy للتطبيق وGitLab. عنوان الاستخدام النهائيان هما:

https://foxflow.20-127-65-116.sslip.io
https://gitlab.20-127-65-116.sslip.io

قواعد Azure NSG تسمح بـ 80 و 443 و GitLab SSH على 2222. بقيت 3000 و 5050 و 8080 و 8443 خاصة، ولذلك لا يصل المستخدم إلى الخدمات الداخلية مباشرة.

النسخ الاحتياطي والاستعادة

أكملت التكامل التشغيلي للنسخ الاحتياطي: GitLab ينشئ ملف tar، ويرفعه إلى container خاص في Azure Blob Storage. يوجد cron يومي الساعة 03:00، وتفحص المراقبة عمر النسخة. كما يوجد verify-backup.sh لتنزيل أحدث نسخة وفحص سلامتها وrestore.sh لتنفيذ الاستعادة عند الحاجة.

القيمة	الدليل
gitlab_backup.tar_030001_20260926	اسم آخر نسخة موثقة
82,370,560 بايت	الحجم
Azure Blob ffbakupstorage01/gitlab-backups	الموقع الخارجي
نجح	تنزيل النسخة
نجح	فحص tar -tf
يوميًا 03:00	الجدولة

لم تُنفذ استعادة فعلية على GitLab الإنتاجي لأن الاستعادة تستبدل قاعدة البيانات والمستودعات الحالية. جرى توثيقها كاختبار مستقبلي على VM منفصلة، وهو الخيار الآمن.

تجربة الفشل الرسمية

جهزت سيناريو عرض يعطل نسخة تجريبية منفصلة على منفذ داخلي بدل تعطيل الإنتاج. تعيد النسخة 503 وتصيح unhealthy، فتلتقط المراقبة الحالة وترسل تنبيهًا. عند recover تعود 200 وتصل رسالة التعافي. يبقى التطبيق الحقيقي على الإنتاج يعمل طوال التجربة.

النتيجة	الأمر
إنشاء نسخة تجريبية غير سليمة وإرسال إنذار	fail
عرض الحالة والجسم ورمز HTTP	status
إعادة النسخة إلى healthy وإرسال تعاف	recover
حذف الحاوية والملفات المؤقتة	cleanup

```
bash scripts/committee-failure-demo.sh fail
bash scripts/committee-failure-demo.sh status
bash scripts/committee-failure-demo.sh recover
bash scripts/committee-failure-demo.sh cleanup
```

المشكلات التي واجهتها وكيف حُلَّت

الحل	السبب	المشكلة
استخدام Registry الداخلي 10.0.1.4:5050 وضبط dind	Runner يصل من شبكة Docker مختلفة	Registry Unauthorized

Azure Read Only	وجود اشتراك آخر معطل بالاسم نفسه	اختيار الاشتراك النشط الذي يبدأ بـdd4c4482
فتح Registry للعامه	المنفذ لا يحتاج وصول المستخدمين	إبقاء Registry داخليًا وعدم كشف 5050
blob unknown	Registry لا يدعم attestation الحديثة	BUILD_X_NO_DEFAULT_ATTESTATION_S=1
فشل Trivy	حزمة tar التابعة لـnpm كانت متأثرة	حذف npm وnpmx من Runtime
Connection refused	GitLab وRegistry كانا في مرحلة starting	انتظار health=healthy ثم إعادة الفحص
not a git repository	تنفيذ commit من مجلد غير صحيح	تشغيل Git من جذر SDA-project- FoxFlow
Mac على docker ps	Docker يعمل في الخادم وليس Mac	الدخول إلى Azure VM عبر SSH
تنبيه مراقبة خاطئ	فحص HTTP القديم أعطى إنذارًا كاذبًا	استخدام مسار GitLab مناسب ومراقبة التعافي
رسالة AI دون تشخيص	رد OpenRouter لم يكن دائمًا بالصيغة المتوقعة	إضافة fallback محلي وتنقية الرد
غياب HTTPS	الحاجة إلى اتصال موثوق بدل HTTP	Caddy وsslip.io وتقييد المنافذ

أهم الملفات التي أنشأتها أو عدلتها

الوظيفة	الملف
تعريف مراحل البناء والاختبار والفحص والنشر والإشعار	sample-app/.gitlab-ci.yml
النشر الآمن عبر SSH وCompose	sample-app/ci/deploy.sh
إشعارات Telegram وتحليل OpenRouter	sample-app/ci/notify.mjs
تطبيق FoxFlow وAPI والمصادقة والصحة	sample-app/server.js
اختبارات التطبيق واللوحة	sample-app/test.js
لوحة Control Room	sample-app/public/dashboard.html
الصفحة العامة وصفحة الدخول	login.html و sample-app/public/index.html
قواعد تشغيل الإنتاج والصحة	sample-app/deploy/docker-compose.yml
HTTPS reverse proxy	sample-app/https/Caddyfile
المراقبة والخدمة والجدولة	*/sample-app/monitoring
Copilot والأدوات والاختبارات	*/sample-app/copilot

scripts/committee-failure-demo.sh و */sample-app/demo	تجربة الفشل
restore.sh و verify-backup.sh و scripts/backup.sh	النسخ والتحقق والاستعادة
scripts/publish-sample.sh	نشر sample-app إلى GitLab
integration-test-plan.md و docs/ci-cd.md	توثيق CI/CD والأدلة
architecture-presentation- و *-docs/foxflow-architecture script-ar.md	مخططات معمارية وسكرت شرح

أدلة الإنجاز النهائي

القيمة الموثقة	الدليل
Pipeline #44	آخر Pipeline ناجحة
deploy و security_scan و copilot_test و test و build و notify_success و	الوظائف الناجحة
16/16	الاختبارات
Critical أو High 0	بوابة Trivy
{ "status": "ok", "service": "foxflow-sample" } و HTTP 200	فحص الصحة
healthy	foxflow-app-app-1
healthy	foxflow-gitlab
active	foxflow-monitor.timer
active	foxflow-copilot.service
26 سبتمبر 2026	آخر Backup verified
HTTPS موثوق	التطبيق و GitLab

مطابقة المشروع مع المطلوب الأصلي

الحالة	الدليل	البند
مكتمل	Terraform resources موجودة وتعمل	Azure provisioning
مكتمل	GitLab CE على Docker Compose	GitLab deployment
مكتمل	Runner و Pipeline فعلية	CI/CD demonstration
مكتمل	سكربتات تثبيت وتشغيل وفحص ونسخ	Bash automation

Setup templates	قوالب env و tfvars	مكتمل
Documentation	README ووثائق التشغيل والتعافي	مكتمل
Merge Request و Branch	المنصة تدعمه لكن يفضل MR تجريبية موثقة	جزئي ويحتاج دليل عرض
OS hardening	SSH و HTTPS موجودة؛ يمكن إضافة hardening لـ Ubuntu	جزئي

ما بقي كتحسين مستقبلي

- إنشاء Feature Branch و Merge Request موثقة للمراجعة أمام اللجنة.
- تجربة Restore كاملة على Azure VM منفصلة.
- نسخ gitlab.rb و gitlab-secrets.json بطريقة مشفرة.
- إضافة Azure lifecycle policy للاحتفاظ بالنسخ القديمة وحذفها.
- إضافة Rollback تلقائي إذا فشل فحص الصحة بعد النشر.
- توثيق تقوية Ubuntu مثل تعطيل SSH password و root login والتحديثات الأمنية.
- تحديد RTO و RPO رسميًا.

طريقة عرض المشروع أمام اللجنة

1. ابدأ من المخطط المعماري و اشرح أن GitLab هو منصة العمل الرئيسية و GitHub نسخة أرشيفية.
2. افتح التطبيق عبر HTTPS ثم افتح health/ وأظهر HTTP 200 وحالة ok.
3. افتح لوحة العمليات وأظهر آخر Pipeline ومراحلها وحالة Runner و Trivy.
4. نفذ تعديلًا صغيرًا أو اعرض Merge Request ثم ادفعه إلى main المحمي.
5. تابع Pipeline وهي تنفذ Build و Test و Security و Deploy و Notify.
6. أظهر وصول رسالة النجاح إلى Telegram وتحديث اللوحة.
7. نفذ تجربة fail المعزولة و اعرض 503 والتنبيه والتحليل، مع إثبات استمرار الإنتاج.
8. نفذ recover ثم cleanup و اعرض رسالة التعافي.
9. اختتم بإثبات النسخة الاحتياطية في Azure واذكر أن restore drill تحسين تالي.

نص مختصر لشرح دوري

كنت مسؤولاً عن تحويل الكود إلى خدمة تُبنى و تُختبر و تُفحص و تُنشر تلقائيًا. صممت GitLab Pipeline من خمس مراحل، وربطت Runner بالـ Registry الداخلي. بيني النظام صورة Docker مرتبطة بالـ commit، ثم يشغل 16 اختبارًا ويفحصها بواسطة Trivy. إذا نجحت البوابات، ينشر GitLab الصورة إلى Azure عبر SSH و Docker. Compose و ينتظر حتى تصبح الخدمة healthy.

بعد النشر، تعرض لوحة FoxFlow الحالة الحقيقية، وترسل Telegram نتيجة Pipeline. عند الفشل يقرأ OpenRouter السجل و يشرح السبب والدليل والحل، مع تحليل محلي بديل إذا تعذر AI. كما أضفت مراقبة دورية و AI Copilot ونسخًا احتياطيًا إلى Azure و HTTPS وحماية للوحة بحسابات منفصلة.

وللعرض أمام اللجنة جهزت فشلًا معزولًا يعيد 503 ويرسل إنذارًا دون تعطيل الإنتاج، ثم يعيد recover الخدمة إلى 200 ويرسل رسالة تعاف. آخر تحقق هو Pipeline #44، والاختبارات 16 من 16، ولا توجد ثغرات High أو Critical تمنع النشر.

أسئلة متوقعة وإجاباتها

لماذا GitLab وليس GitHub؟

GitLab هو المنصة الداخلية الرئيسية ويجمع المستودع والمراجعات والـRegistry والـRunner والـCI/CD. يحتفظ بنسخة أرشيفية فقط.

كيف تمنعون نشر كود فاشل؟
Deploy تعتمد على نجاح Test و Copilot Test و Trivy، ثم تنتظر healthcheck. أي فشل في بوابة إلزامية يوقف النشر.

هل Registry مكشوف للعامة؟
لا. يستخدمه GitLab Runner والخادم داخل الشبكة الخاصة، ولا تسمح NSG بالوصول العام إلى 5050.
أين تحفظ الأسرار؟
في GitLab CI/CD Variables المحمية والمخفية، أو ملفات بصلاحيات 0600 على الخادم. لا تُحفظ قيم الأسرار في Git.

ماذا يحدث إذا تعطل OpenRouter؟
يعمل تحليل محلي آمن وتصل رسالة مفيدة بدل فقدان التنبيه.
هل تجربة الفشل توقف الإنتاج؟
لا. تعمل في حاوية معزولة، ويظل تطبيق الإنتاج على حالته الطبيعية.
هل النسخ الاحتياطي مجرّب؟
تم إنشاء النسخة ورفعها وتنزيلها وفحص سلامة الأرشيف. الاستعادة الكاملة مؤجلة إلى VM منفصلة لأنها عملية مؤثرة.

روابط وأوامر مرجعية

العنوان	الخدمة
https://gitlab.20-127-65-116.sslip.io	GitLab
https://foxflow.20-127-65-116.sslip.io	التطبيق
https://foxflow.20-127-65-116.sslip.io/dashboard	لوحة العمليات
https://foxflow.20-127-65-116.sslip.io/health	فحص الصحة

```
cd ~/SDA-project-FoxFlow/sample-app
npm test
python3 -m unittest copilot.test_ai_copilot -v

curl -i https://foxflow.20-127-65-116.sslip.io/health

ssh -i ~/.ssh/foxflow azureuser@20.127.65.116
docker ps
systemctl status foxflow-monitor.timer
systemctl status foxflow-copilot.service
```

الخلاصة

أعمال FoxFlow غطت التكامل المستمر والنشر الآلي والأمن التشغيلي والمراقبة والتنبيهات الذكية ولوحة الحالة وحماية الوصول وHTTPS وتجهيز العرض. المشروع يحقق شروط القبول الأساسية، وأصبح قابلاً للتشغيل والتوضيح أمام الفريق واللجنة بأدلة رقمية حقيقية بدل الاكتفاء بوصف نظري.
بعد أي تحسين جديد، حدّث رقم Pipeline وتاريخ فحص النسخة الاحتياطية في قسم الأدلة.